Benchmark-Anchored Failure Regimes in Multi-Agent Systems: A Topology-Controlled Trace Audit

Anonymous Authors[1]

Abstract

Multi-agent topologies are often evaluated as if additional agents were a general-purpose capability amplifier. We test that assumption with a trace-centered audit across five benchmarks and seven coordination designs using MAS Analyzer, our experimental framework for running and diagnosing multi-agent benchmark suites. The completed corpus contains 34 benchmark-by-design summaries, 3,060 evaluated runs, 990 task summaries, and 2,425 failed runs with trajectory and evaluation artifacts. Our central finding is that failure regimes are benchmark-anchored: benchmark identity largely determines the dominant failure kind, while topology mostly changes the probability of escaping that regime. Group-chat debate substantially improves StableToolBench; simple voting is strongest on BrowseComp, PlanCraft, and WorkBench; orchestration barely moves Finance Agent. These gains are often purchased at 4–14× token cost with little failure-regime change outside StableToolBench. Three trace diagnostics—tool-call Markov transitions, vote entropy, and first-critical-fault localization—confirm the mechanism and convert failure labels into reproducible repair hypotheses.

1. Introduction

Agentic systems increasingly solve tasks by interleaving reasoning, tool use, feedback, communication, and environment interaction (Yao et al., 2023; Schick et al., 2023; Wu et al., 2023). They are also increasingly evaluated through benchmark success rates (Liu et al., 2023; Zhou et al., 2023; Guo et al., 2024), but terminal scores hide the process by which a run fails. A final answer can be wrong because retrieval locked onto a weak hypothesis, because a planner declared impossibility after a local shortage, because an API returned sparse evidence, or because a workflow never grounded a required person or email address. These are not interchangeable errors: each implies a different repair.

This paper studies whether multi-agent topology changes those failures. The question is deliberately narrower than "do more agents help?" Multi-agent systems and debate suggest that multiple agents can critique, complement, or coordinate with one another (Li et al., 2023; Du et al., 2024), but this does not imply that coordination changes the underlying failure mechanism in long-horizon agent settings. We ask when topology changes success, when it only changes cost, and when the same benchmark-specific failure regime persists across coordination structures. We analyze a MAS Analyzer experiment suite spanning BrowseComp, PlanCraft, StableToolBench, WorkBench, and Finance Agent. The evaluated systems include a single-agent baseline, simple voting, fully linked debate, group-chat debate, and three orchestrator variants.

Our central finding is that failure regimes are topology-sensitive in rate but benchmark-anchored in kind: changing topology often changes how often a system escapes a regime, but rarely changes the dominant regime itself. The strongest topology differs by benchmark, and the largest success gain appears only on the tool-heavy StableToolBench. We view an agent run as a sequence of state transitions; a failure regime is the recurring pattern by which an early corrupted state propagates into a terminal failure.

The paper makes four contributions. First, it reports a cross-benchmark topology matrix over 34 completed benchmark-by-design settings run through MAS Analyzer, our open experimental framework for deploying, tracing, and evaluating multi-agent systems across benchmark suites. Second, it codes 2,425 failed runs into operational failure regimes using evaluation JSON,

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country.AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

trajectory markdown, and representative trace inspection. Third, it localizes failed runs to their first trace-visible bottleneck, separating the earliest corrupted state from the terminal failure label. Fourth, it converts each failure regime into a repair hypothesis with trace-local diagnostics: first-critical-fault localization for tracing terminal failures back to their earliest visible bottleneck, tool-call Markov transitions for tool-heavy tasks, and vote entropy for consensus health. We treat these diagnostics as falsifiable repair hypotheses; intervention verification is left to future work.

2. Related Work

Reasoning, acting, and tool use in language agents. Recent agent frameworks extend language models from static answer generation to iterative interaction with tools, environments, feedback, and accumulated experience. ReAct couples reasoning traces with environment actions (Yao et al., 2023); Toolformer studies learned API invocation (Schick et al., 2023); Reflexion improves agents through verbal feedback and episodic memory (Shinn et al., 2023); and Voyager shows how agents can accumulate reusable skills through open-ended interaction (Wang et al., 2023). AutoGen generalizes this systems view to multiple conversable agents with programmable interaction patterns (Wu et al., 2023). These works motivate the trace artifacts and coordination topologies we study. Our focus is different: we ask when these interaction patterns change the failure mechanism, and when they merely add deliberation around the same corrupted state.

Multi-agent coordination and debate. Multi-agent methods are often motivated by the hope that multiple model instances can check, critique, or complement one another. CAMEL studies role-playing communicative agents (Li et al., 2023), while role-specialized frameworks such as MetaGPT organize agents into structured collaborative workflows (Hong et al., 2023). Multi-agent debate shows that several model instances can improve factuality and reasoning through proposal and critique (Du et al., 2024). These results suggest that coordination can help, but not that it is uniformly beneficial in long-horizon agent settings. Our results qualify this assumption: topology is useful when it changes the relevant state variable, such as tool evidence coverage, but can become redundant confirmation when all agents inherit the same weak evidence, premature impossibility state, or unresolved entity binding.

Agent benchmarks and long-horizon evaluation. A growing set of benchmarks evaluates agents in interactive and tool-mediated environments. AgentBench evaluates LLM agents across diverse interactive settings (Liu et al., 2023), and WebArena provides realistic web environments for autonomous agents (Zhou et al., 2023). ToolBench and StableToolBench focus on large-scale tool-use evaluation, with StableToolBench reducing API-status instability through a virtual API server and stable protocol (Qin et al., 2023; Guo et al., 2024). More specialized benchmarks expose different bottlenecks: BrowseComp stresses hard-to-find web evidence (Wei et al., 2025); PlanCraft evaluates planning, tool use, and solvability judgments in a Minecraft-inspired crafting environment (Dagan et al., 2024); TheAgentCompany provides workplace-style tasks involving browsing, coding, execution, and communication, which we refer to as WorkBench in our suite (Xu et al., 2024); and Finance Agent Benchmark evaluates tool-assisted financial research over filings and market information (Bigeard et al., 2025). Rather than introduce another benchmark, we use a common trace format to compare how these benchmark families interact with coordination topology.

Trace diagnostics and failure localization. Most benchmark reporting emphasizes terminal success, but agent runs expose richer intermediate artifacts: reasoning traces, tool calls, vote tallies, summaries, and environment responses. Long-horizon failures often become trace-visible before the final answer, when evidence stops expanding, tool results become low-information, planners promote local shortages into global impossibility, or workflows remain blocked by unresolved entities. Recent work derives multi-agent failure taxonomies from execution traces (Cemri et al., 2025); our work is complementary in asking how topology changes the distribution, cost, and recoverability of benchmark-grounded failure regimes. We connect terminal failures to first-critical-fault localization, tool-state transitions, vote entropy, and coordination-tax proxies, positioning trace diagnostics as reproducible triggers and repair hypotheses rather than post-hoc anecdotes.

3. Corpus and Method

3.1. Coordination Designs

Topology is the experimental variable of interest. We compare seven coordination designs, listed in Table 1 with the short labels used in all figures throughout this paper.

Short	Full name	Description
SAS	Single-agent baseline	One agent; no coordination.
Vote	Simple voting	N agents propose independently; majority vote selects.
Full	Fully linked debate	All agents debate in a fully connected graph.
Chat	Group-chat debate	Agents debate via a shared group-chat channel.
O-no	Orchestrator (no disc.)	Orchestrator delegates; subagents do not discuss.
O-tree	Orchestrator (tree)	Tree-structured task decomposition.
O-disc	Orchestrator (+disc.)	Orchestrator with post-delegation agent discussion.

Table 1. Topology abbreviations used in all figures and tables.

3.2. Experiment Scope

The suite contains five benchmarks: BrowseComp, PlanCraft, StableToolBench, WorkBench (TheAgentCompany), and Finance Agent. One aggregate topology setting is absent from the completed results, leaving 34 benchmark-by-design pairs. Each completed pair has 30 tasks and three repeated runs, yielding 3,060 evaluated runs. Task-level summaries cover 990 benchmark tasks over 33 settings.

3.3. Metrics

We report task success, stability across repeated runs, average token use, tool calls, communication count, and handoff count. For topology comparisons, we compute success gain relative to the single-agent baseline (SAS) within the same benchmark. The token multiplier is each topology's average total tokens divided by the SAS average for the same benchmark.

3.4. Trace Artifacts

The audit is possible because the runtime standardizes intermediate agent outputs. Each answer-producing stage emits an answer artifact, summary, critique, revision request, confidence, unresolved issues, and evidence summary. The trajectory files also record task prompts, tool definitions, role assignments, visible peer packets, final reasons, and vote tallies when applicable. This structure is important methodologically: it lets us separate an unsupported final answer from an explicit blocked answer, and it lets us ask whether a topology introduced new evidence or merely recirculated the same artifact.

We treat the benchmark evaluator output as the outcome source of truth. The trace is then used to explain the failure path, not to override the evaluator. This distinction prevents fluent but wrong answers from being credited simply because they look coherent, and prevents honest blocked answers from being collapsed into generic model error.

3.5. Failure Coding

We code every failed run in the 34 completed settings, for 2,425 failures in total. Coding begins from the benchmark evaluator output, then uses the prediction string, final reason, trajectory markdown, vote tally, visible packets, tool outputs, and representative event traces to assign one primary label. The labels are operational rather than universal. They are benchmark-grounded by design; their value lies in the trace-local diagnostic each label induces. For example, PlanCraft failures are coded as premature impossibility when the run emits impossible after a local shortage; WorkBench failures are coded as entity grounding failure when a missing person, sender, email, or assignee blocks execution.

When multiple mechanisms appear in the same run, we use an earliest-bottleneck rule: the primary label is the first trace-visible condition that makes the benchmark objective unattainable under the observed run. If a sparse API response later leads to an unsupported answer, the run is coded as tool/interface fragility when the tool layer is the binding constraint, and as unsupported synthesis when the tools return usable evidence but the final synthesis overstates it. If a workflow later stalls because an email address was never grounded, the run is coded as entity grounding rather than generic workflow blocking.

This is a lightweight trace audit, not a multi-rater annotation study. Its purpose is diagnostic: to identify whether failures across topologies share the same trace-visible mechanism, and whether topology changes the distribution of those mechanisms. We therefore prefer labels with clear repair implications over labels that merely restate the benchmark outcome.

3.6. State-Transition Diagnostics

We add three trace diagnostics on top of the primary labels. For StableToolBench, we parse all non-communication tool-result events from run trace files and collapse each result into one of three states: OK, sparse, or error. We then estimate row-normalized Markov transitions separately for successful and failed runs. Table 2 gives the reproducible state rules. Observed status strings include completed, ok, and er-

State	Rule	Examples
OK	Non-empty structured response, no error cue, and preview length at least 20 characters.	Valid JSON fields; populated records.
sparse	Empty or degenerate preview without an explicit error cue.	Empty list, empty object, null, no data found.
error	Status or preview contains interface-failure cues.	Cache miss, invalid parameters, exception, timeout, rate limit.

Table 2. Tool-result state rules used for StableToolBench Markov transitions. If multiple cues appear, error takes precedence over sparse.

ror; status values error, failed, or timeout are always treated as error.

For voting topologies, we compute Shannon entropy over the recorded vote tally. Let p_i be the share of recorded votes assigned to option i; then $H = -\sum_i p_i \log_2 p_i$, normalized by the maximum possible entropy for the number of observed options. This is a diagnostic of recorded consensus concentration, not a measure of correctness.

For the full failed-run corpus, we also compute a first-critical-fault label. This is not a claim of formal causal identification. It is a rule-based diagnostic attribution that maps each failed run to the earliest visible bottleneck class implied by evaluator output and trace artifacts: retrieval evidence gap, unsupported hypothesis, premature impossibility, tool/interface fault, entity grounding, workflow precondition, data-interface brittleness, or financial support gap. The first-critical-fault label differs from the primary failure label: the primary label explains the failed outcome, while first-critical-fault localizes the earliest trace-visible state that contaminates the rest of the run. We apply this diagnostic to the 34 completed settings in the experiment summary. As a sanity check, we re-audited a deterministic stratified sample of 30 failed runs, six per benchmark, and found 30/30 agreement with the rule output. This is a one-author reliability check rather than an independent multi-rater study.

4. Topology Does Not Have a Universal Winner

Figure 1 shows the central result. Vote is strongest on BrowseComp, PlanCraft, and WorkBench; Chat (group-chat debate) is strongest on StableToolBench; and O-tree is strongest on Finance Agent. The best topology is therefore benchmark-dependent, not a general property of agent count.

Benchmark	Best design	Best	Single agent	Gain
BrowseComp	Simple voting	.111	.078	+.033
PlanCraft	Simple voting	.378	.367	+.011
StableToolBench	Group-chat debate	.567	.333	+.233
WorkBench	Simple voting	.211	.167	+.044
Finance Agent	Tree orchestrator	.067	.044	+.022

Table 3. Best coordination design by benchmark. The largest success gain appears on StableToolBench, while other benchmarks show smaller topology effects.

Table 3 shows that topology effects are uneven. StableToolBench gains 23.3 points over the single-agent baseline, while PlanCraft gains only 1.1 points and Finance Agent gains only 2.2 points. These results already suggest that coordination is useful only when it changes the benchmark's bottleneck.

5. Failure Regimes Are Benchmark-Anchored

The central question in this section is whether changing coordination topology also changes the kind of failure a run encounters. We approach this in two passes: first, by localizing the earliest trace-visible bottleneck in each failed run (the first-critical-fault view); then, by examining the terminal failure label that best explains the final outcome. The two views are complementary—they answer different questions—and together they establish that benchmark identity, not topology, determines the dominant failure mechanism.

5.1. First Critical Faults

We ask where a failed run first becomes unrecoverable. The first-critical-fault view localizes the earliest visible bottleneck before the terminal answer or evaluator label. This matters because the final symptom can be misleading: an unsupported final answer may begin as a retrieval gap, and a blocked workflow may begin as an unresolved person or email address.

Figure 2 shows that these earliest bottlenecks are sharply benchmark-specific. BrowseComp failures begin when retrieval fails to expand evidence or when a weak candidate becomes the working hypothesis. PlanCraft failures begin when a local resource shortage is promoted to a global impossible judgment. StableToolBench failures begin when tool responses become sparse, schema-limited, or interface-constrained. WorkBench failures begin when a required entity remains ungrounded before an irreversible workflow action. Finance Agent failures begin when financial claims lack period-aligned filing support or when the data interface becomes brittle.

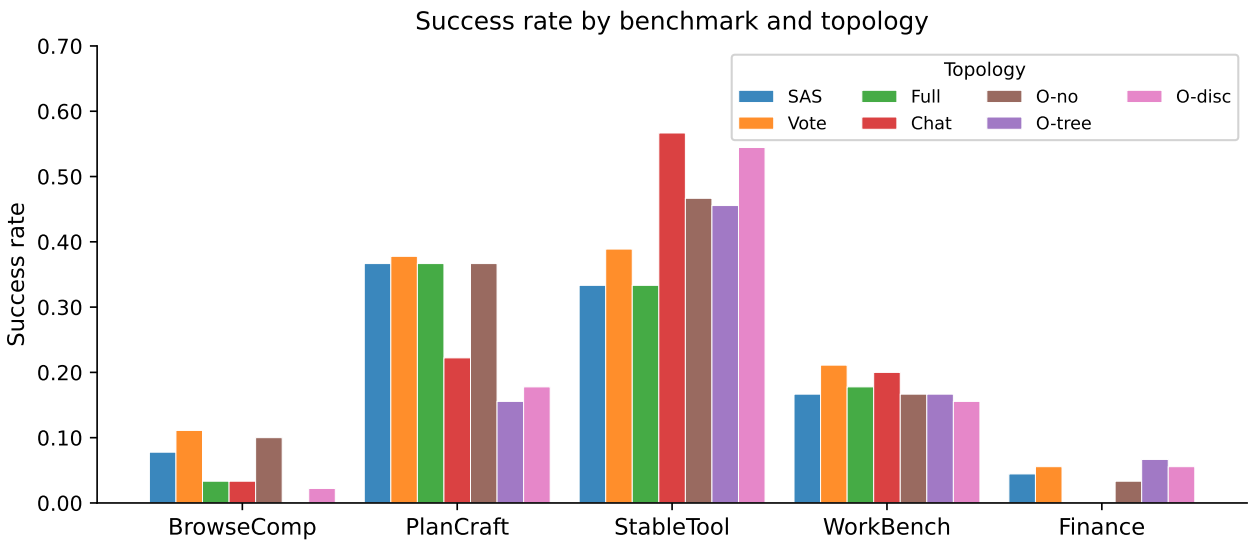

Figure 1. Success rate by benchmark and topology (see Table 1 for abbreviations). No topology dominates across benchmarks; topology effects are largest on StableToolBench and near-flat on Finance Agent and PlanCraft.

Quantitatively, WorkBench localizes first to entity grounding in 364 of 518 failures (70.3%), while PlanCraft localizes to premature impossibility in 445 of 447 failures (99.6%). StableToolBench localizes to tool/interface faults in 286 of 352 failures (81.3%). Finance Agent splits between financial support gaps (353 of 512, 68.9%) and data-interface brittleness (159 of 512, 31.1%). BrowseComp is dominated by unsupported hypothesis promotion (441 of 596, 74.0%), with 155 failures (26.0%) showing explicit retrieval evidence gaps.

Appendix Figure 9 summarizes the same point as a propagation view. Retrieval gaps and unsupported hypotheses feed contaminated evidence states, premature impossibility feeds invalid terminal states, and grounding failures feed unresolved action bindings. Useful interventions should therefore target the earliest state transition that contaminates the rest of the run.

5.2. Terminal Failure Labels

Having established where failures originate, we now examine what they look like at the end. Figure 3 reports primary failure labels: the terminal regime that best explains why a run failed.

BrowseComp is dominated by unsupported hypotheses: 487 of 596 failed runs return an answer whose support chain is too weak, while 109 explicitly report evidence starvation. PlanCraft is the cleanest regime: 444 of 447 failures are premature impossibility. StableToolBench splits between unsupported synthesis (180) and tool/interface fragility (172). WorkBench failures are mostly grounding related: 304 entity grounding failures and 214 broader workflow blocks. Finance Agent is dominated by financial support gaps (430 of 512), with 82 stack or data brittleness cases.

The important point is not that these labels are perfect. The important point is that topology rarely erases the benchmark's dominant regime. In BrowseComp, coordination-heavy systems still fail by building around weak evidence. In PlanCraft, debate and orchestration do not prevent the local-to-global impossibility jump. In WorkBench, additional agents cannot execute a workflow if none of them resolves the missing entity. StableToolBench is the main exception: because its bottleneck involves assembling partial tool evidence, group discussion can change the run's chance of success.

We use labels as repair hypotheses, not merely as names. Table 4 pairs each label with a trace-visible state variable, a topology implication, and a repair-oriented diagnostic. The topology-conditioned heatmaps in Appendix Figure 8 support the same claim without aggregating away topology: within most benchmarks, the dominant failure column is stable across topology rows even as success rates vary.

6. Topology Changes Cost Before It Changes Failure

Table 5 shows the token multiplier and success gain for the best-performing topology on each benchmark.

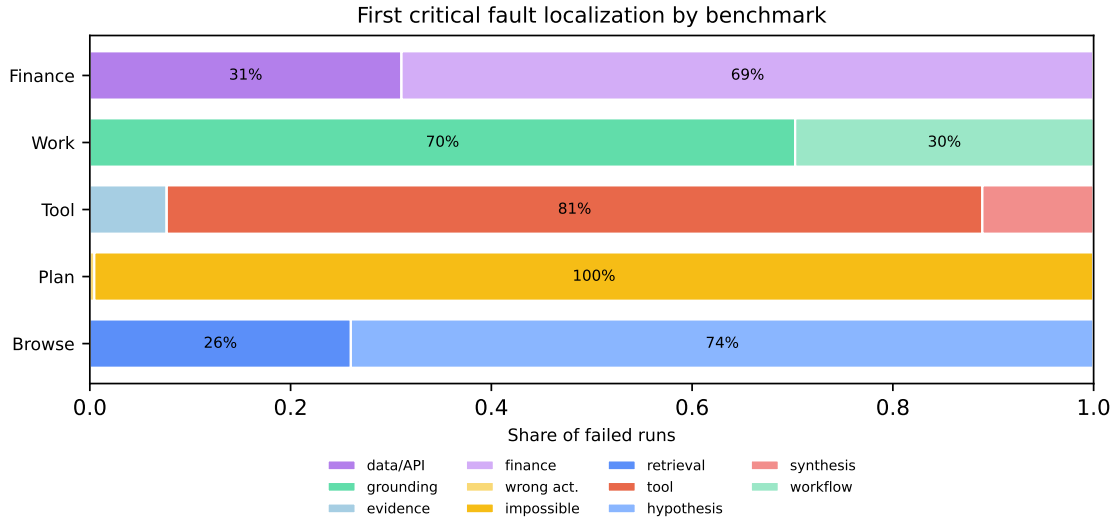

Figure 2. First-critical-fault localization over failed runs. The earliest visible bottleneck is benchmark-specific and largely stable across topologies, supporting the claim that topology operates on top of benchmark-anchored failure regimes.

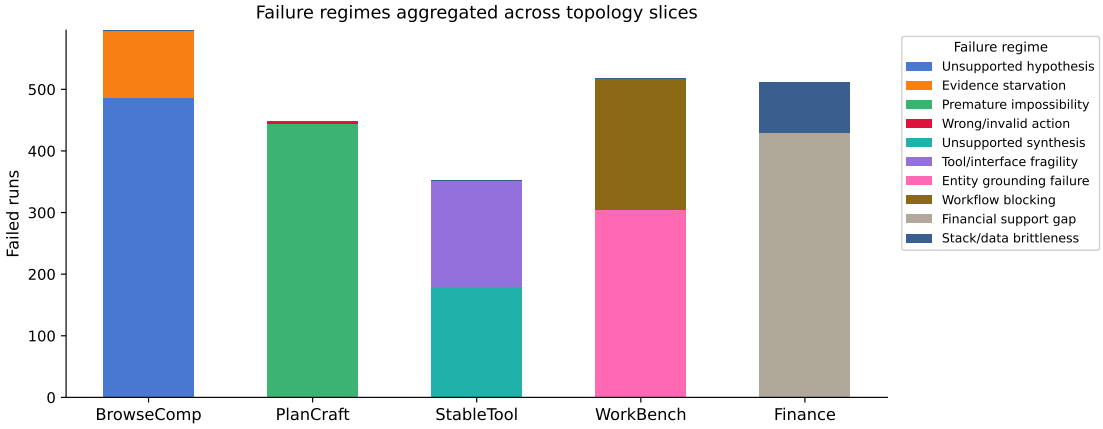

Figure 3. Failure labels aggregated across all completed topology slices. The dominant failure regime is largely determined by benchmark family, not coordination topology.

Coordination almost always costs 3–15× more tokens than the single-agent baseline. The gain is substantial only on StableToolBench, where group-chat debate adds 23.3 points at 9.7× token cost. Elsewhere the picture is unfavorable: the best topology on BrowseComp (Vote, +3.3 points) costs 4.3×, and on PlanCraft (Vote, +1.1 points) costs 3.9×. Orchestration with discussion is the worst case: on PlanCraft it spends 14.5× tokens for a 18.9-point loss.

This cost asymmetry matters for evaluation practice. A system that gains two points at 10× token cost is not an unqualified improvement over the single-agent baseline. Future reporting should therefore accompany topology comparisons with token multipliers so that downstream users can judge whether the coordination benefit is worth the inference budget. We return to this in Section 9.

7. Trace Diagnostics Explain the Mechanism

Having established that failure regimes are benchmark-anchored (Section 5) and that coordination gains are often purchased at disproportionate cost (Section 6), we now examine the trace-level mechanism behind both findings. Three diagnostics confirm why topology shifts success probabilities without shifting failure kinds.

7.1. Tool-Call Markov Transitions

StableToolBench is the positive control for coordination, so we inspect its tool-state dynamics directly. Figure 4 compares row-normalized transitions between

Benchmark	Dominant trace mechanism	Topology implication	Repair-oriented diagnostic
BrowseComp	Evidence state remains thin while candidate answers become increasingly salient.	Debate can amplify weak hypotheses unless agents seek disconfirming evidence.	Unique evidence growth and candidate-disconfirmation checks before final vote.
PlanCraft	Local resource shortage is promoted to global impossibility.	More agents help only if one preserves the unexplored frontier.	Count alternate route expansions before impossible; require frontier-exhaustion proof.
StableToolBench	Sparse or schema-limited API responses constrain downstream synthesis.	Discussion helps when agents can combine complementary tool outputs.	Low-information response streaks, API coverage gaps, and synthesis support density.
WorkBench	Workflow cannot bind a required person, email, sender, or assignee.	Coordination is orthogonal unless an agent resolves the binding.	Unresolved entity count before first irreversible workflow action.
Finance Agent	Financial claims exceed verified filings, definitions, or extracted numeric support.	Orchestration often cannot repair missing evidence.	Source coverage, period alignment, formula completeness, and citation-to-claim density.

Table 4. Automated audit blueprint. Each failure regime is tied to a trace-visible state variable, a topology-specific implication, and a repair-oriented diagnostic. Repeated search with overlapping snippets becomes a unique-document-growth check; early impossible after a local shortage becomes a frontier-expansion check; repeated sparse tools become a low-information-streak check; missing entities become unresolved-binding checks; and thin numerical support becomes source-coverage and period-alignment checks.

Benchmark	Best topology	Token mult.	Success gain
BrowseComp	Vote	4.3×	+.033
PlanCraft	Vote	3.9×	+.011
StableToolBench	Chat	9.7×	+.233
WorkBench	Vote	3.5×	+.044
Finance Agent	O-tree	8.4×	+.022

Table 5. Token multiplier relative to SAS and success gain for the best topology per benchmark. Only StableToolBench shows a gain that plausibly justifies its cost.

coarse tool-result states for successful and failed runs. The largest separation is not the probability of remaining in an error state, which is high for both outcomes. Instead, it is the probability that an apparently usable tool state deteriorates on the next tool result. In successful runs, OK→sparse/error occurs with probability 0.184. In failed runs, the same deterioration occurs with probability 0.298, a 1.62× increase.

This matters because it reframes tool failure as a transition property rather than a terminal label. A single sparse response is not necessarily fatal; many successful runs also encounter imperfect tool outputs. The dangerous pattern is repeated deterioration without a strategy shift. Failed StableToolBench runs have a higher mean maximum adverse streak than successful runs (2.76 vs. 2.17), while using nearly the same mean number of tool results. The bottleneck is therefore not simply tool volume. It is whether the run can recover after the tool state becomes low-information.

7.2. Consensus Is Not Verification

Vote entropy explains why topology can fail without visible disagreement. A group may look confident not because it verified the answer, but because all agents inherited the same contaminated state. Appendix Figure 10 asks whether voting actually exposes disagreement. Across 1,236 runs with recorded vote tallies, consensus is often highly concentrated. In failed group-chat runs, the mean normalized vote entropy is only 0.084 and 91.6% of tallies contain a single surviving option. Fully linked debate is also concentrated: failed runs have mean normalized entropy 0.279 and a 69.7% single-option rate. Voting is less concentrated, but still far from adversarial: failed runs have mean normalized entropy 0.390 and a 58.6% single-option rate.

The key result is not that low entropy uniquely predicts failure. Successful runs also often have low entropy. Instead, vote entropy shows why consensus is an unsafe proxy for independent verification: agents can converge quickly around either a correct answer or a contaminated evidence state. This is the quantitative counterpart to the PlanCraft case below, where a feasible action appears in the artifact set but the final aggregation still promotes a terminal impossibility state.

8. Case Studies

We present two representative traces: one where coordination promotes an avoidable terminal state (Plan-

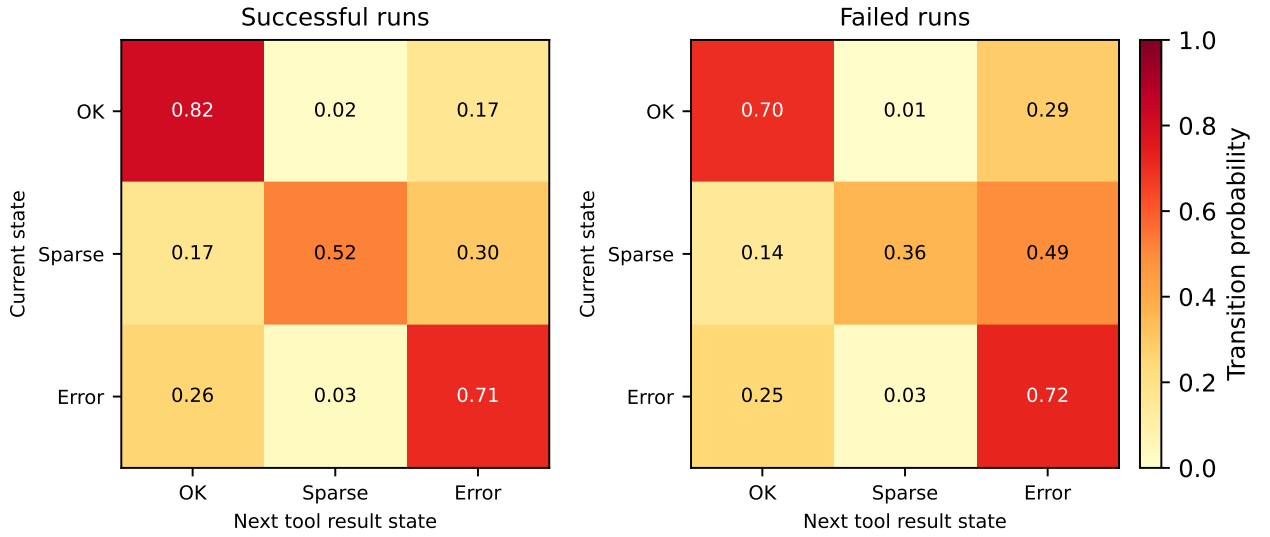

Figure 4. StableToolBench tool-result Markov transitions. Failed runs are more likely to transition from an OK tool result into an adverse sparse/error state on the next tool result.

Craft), and one where coordination has a plausible division-of-labor target (StableToolBench). These two benchmarks were chosen because they represent opposite ends of the topology-effect spectrum—PlanCraft shows near-zero topology benefit with a single dominant failure regime, while StableToolBench shows the largest topology benefit via a genuinely collaborative repair mechanism. Additional benchmark notes appear in the appendix.

8.1. PlanCraft: Local Shortage Becomes Global Impossibility

One PlanCraft trace contains a feasible frontier-expanding action in the candidate artifacts, and one critique explicitly notes that declaring the task impossible ignores the benchmark rule. Nevertheless, the final answer is impossible, with deterministic voting used to resolve the aggregation state.

This is a sharper failure than a generic planning miss. A valid route is visible in the artifact set, but the aggregation rule resolves a tie toward a terminal action. The fatal pivot is therefore a topology-mediated state transition: candidate actions include a feasible frontier-expanding step, yet finalization promotes the premature impossibility state. This case shows why voting is not automatically robust. It can reduce conversational amplification, but it still needs a domain-aware preference rule that ranks reversible frontier-expanding actions above irreversible impossible claims unless impossibility has been proven.

8.2. StableToolBench: API Schema Mismatch

One StableToolBench trace requires combining an ordering constraint with fields that are exposed through different endpoint patterns. In a fully linked debate failure, the available ordered endpoint satisfies part of the request, while other endpoints expose complementary access paths. The final vote favors a blocked answer: the tools cannot satisfy all requested constraints in one direct call.

This trace is not merely a model reasoning failure. The tool surface creates a join problem: the desired answer requires fields and ordering guarantees that are not exposed by the same call. A stronger topology can help only if agents can decompose the operation into endpoint coverage, field recovery, and ordering validation. That is why StableToolBench is the benchmark where group chat plausibly pays for itself. The failure mode is still interface fragility, but the repair is genuinely collaborative when different agents explore complementary API paths instead of repeating the same blocked endpoint.

9. Implications for System Design

The audit supports a state-space view of agentic failure. Each retrieval, tool call, or entity lookup updates the latent execution state. A later action may look reasonable while inheriting a corrupted state. BrowseComp failures often inherit a narrowed hypothesis space; PlanCraft failures inherit an invalid impossibility state; WorkBench failures inherit an unresolved entity binding; StableToolBench failures inherit a low-

information tool state.

This framing turns failure labels into falsifiable mechanistic hypotheses. If evidence starvation is the driver, then forcing diverse retrieval or measuring unique evidence growth should change outcomes. If premature impossibility is the driver, then a formal frontier-exhaustion check should reduce false impossible actions. If tool fragility is the driver, causal interventions on tool responses should shift unsupported synthesis rates. The value of traces is that these hypotheses can be tested at the state-transition level, not only at the final-answer level.

The practical implication is that a stronger evaluation suite should log state changes, not only event counts. It is not enough to know that a tool was called; we need to know whether the call changed evidence coverage. It is not enough to know that several agents voted; we need to know whether the vote preferred reversible exploration over premature termination. And it is not enough to know that a financial answer contained citations; we need to know whether each numerical claim is aligned to the correct period, formula, and source.

The results argue against topology selection by leaderboard average alone. A useful multi-agent system should match coordination to failure regime. Tool-heavy tasks benefit from discussion when agents can combine partial external evidence. Search tasks need stagnation detection and adversarial evidence checks. Planning tasks need formal impossibility standards. Workflow tasks need early entity grounding before deeper orchestration. Finance tasks need support-sensitive answer policies that penalize fluent unsupported specificity. We present these as repair hypotheses supported by trace diagnostics, not as verified fixes.

The evaluation protocol should also report coordination-tax proxies. A topology that gains one or two points at 8–10× token cost may be valuable in a high-stakes setting, but it should not be reported as an unqualified improvement. Conversely, a cheap topology with modest gains may be a better default when the dominant failure mode is not repaired by discussion.

10. Threats to Validity

This study analyzes one experiment batch and one model family. The topology effects may change with stronger models, better tool interfaces, or different prompts. The failure labels are primary labels; many runs exhibit mixed mechanisms, especially tool fragility followed by unsupported synthesis. The coding procedure combines rule-based filtering with trace inspection rather than independent multi-rater annotation. The first-critical-fault re-audit is a one-author sanity check, not a full reliability study. Finally, one aggregate topology setting is absent from the completed results, so cost and task-level comparisons use the available completed settings.

11. Conclusion

Across five benchmarks and seven coordination designs, multi-agent coordination is not a universal fix for agentic failure. StableToolBench benefits substantially from group-chat debate, but BrowseComp, PlanCraft, WorkBench, and Finance Agent show much smaller gains or expensive stagnation. The trace audit explains why: benchmark family largely determines the dominant failure regime, while topology mainly changes the cost and probability of escaping it. Future agent evaluation should therefore report topology-sensitive failure regimes, state-transition diagnostics, and coordination-tax proxies alongside terminal success.

Impact Statement

This paper studies failure in agentic AI systems with the goal of improving reliability and transparency. The positive impact is methodological: trace diagnostics can help developers separate unsupported answers from well-supported ones, distinguish tool failures from reasoning failures, and choose coordination structures with clearer evidence. The risk is that failure triggers could be used to construct adversarial tasks or to overfit systems to benchmark artifacts. In finance-style settings, fluent numerical answers with weak support chains are especially concerning because they can appear credible while failing a faithfulness test. We therefore recommend reporting failure taxonomies together with benchmark scope, uncertainty, and the limits of each diagnosis.

References

Bigeard, A., Nashold, L., Krishnan, R., and Wu, S. Finance agent benchmark: Benchmarking LLMs on real-world financial research tasks. arXiv preprint arXiv:2508.00828, 2025. URL https://arxiv.org/abs/2508.00828.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., et al. Why do multi-agent LLM systems fail? arXiv preprint arXiv:2503.13657, 2025. URL https://arxiv.org/abs/2503.13657.

Dagan, G., Keller, F., and Lascarides, A. Plancraft: An evaluation dataset for planning with LLM agents. arXiv preprint arXiv:2412.21033, 2024. URL https://arxiv.org/abs/2412.21033.

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. In International Conference on Machine Learning, 2024. URL https://arxiv.org/abs/2305.14325.

Guo, Z., Cheng, S., Wang, H., Liang, S., Qin, Y., Li, P., Liu, Z., Sun, M., and Liu, Y. StableToolBench: Towards stable large-scale benchmarking on tool learning of large language models. In Findings of the Association for Computational Linguistics: ACL, 2024. URL https://arxiv.org/abs/2403.07714.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352, 2023. URL https://arxiv.org/abs/2308.00352.

Li, G., Hammoud, H. A. A. K., Itani, H., Khizbullin, D., and Ghanem, B. CAMEL: Communicative agents for "mind" exploration of large language model society. arXiv preprint arXiv:2303.17760, 2023. URL https://arxiv.org/abs/2303.17760.

Liu, X. et al. Agentbench: Evaluating LLMs as agents. arXiv preprint arXiv:2308.03688, 2023. URL https://arxiv.org/abs/2308.03688.

Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., and Sun, M. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. arXiv preprint arXiv:2307.16789, 2023. URL https://arxiv.org/abs/2307.16789.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.

Shinn, N., Cassano, F., Berman, A., and Gopinath, A. Reflexion: Language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, 2023. URL https://arxiv.org/abs/2303.11366.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. arXiv preprint arXiv:2305.16291, 2023. URL https://arxiv.org/abs/2305.16291.

Wei, J. et al. BrowseComp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025. URL https://arxiv.org/abs/2504.12516.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Wang, C., et al. Autogen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023. URL https://arxiv.org/abs/2308.08155.

Xu, F. F. et al. Theagentcompany: Benchmarking LLM agents on consequential real world tasks. arXiv preprint arXiv:2412.14161, 2024. URL https://arxiv.org/abs/2412.14161.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023. URL https://arxiv.org/abs/2210.03629.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., et al. Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854, 2023. URL https://arxiv.org/abs/2307.13854.

A. Failure Coding Rubric

- Unsupported hypothesis: the run returns a candidate answer without sufficient supporting evidence.
- Evidence starvation: the run explicitly reports that necessary evidence was not recovered.
- Premature impossibility: the run emits impossible before the search frontier is exhausted.
- Tool/interface fragility: sparse, empty, inconsistent, or unavailable tool output is the primary bottleneck.
- Unsupported synthesis: tools return some evidence, but the final answer exceeds what the evidence supports.
- Entity grounding failure: a person, email, sender, recipient, user, or assignee cannot be resolved.
- Workflow blocking: execution stalls on an operational precondition not captured by entity grounding alone.
- Financial support gap: a financial answer is fluent or numerically specific but weakly supported.
- Stack/data brittleness: finance failures caused by interacting data, interface, and benchmark constraints.

B. First-Critical-Fault Rule Details

The first-critical-fault diagnostic is generated programmatically by the trace-state diagnostics script. The script first filters runs to benchmark-by-design settings present in the experiment summary CSV; this excludes the incomplete aggregate setting from diagnostic counts. It then assigns the earliest bottleneck class using evaluator text plus selected trace checks:

- BrowseComp: explicit blocked, missing, unsupported, or no-evidence language maps to retrieval evidence gap; otherwise a concrete wrong answer maps to unsupported hypothesis.
- PlanCraft: answers containing impossible map to premature impossibility; other wrong actions map to invalid or wrong action.
- StableToolBench: tool error cues or adverse tool-result traces map to tool/interface fault; missing metadata or unresolved subqueries map to evidence gap; remaining failures map to unsupported synthesis.
- WorkBench: sparse company-directory email lookup results or explicit missing-email language map to entity grounding; remaining failures map to workflow precondition.
- Finance Agent: rate-limit, parsing, API, EDGAR, or other interface cues map to data-interface brittleness; remaining failures map to financial support gap.

We manually re-audited the deterministic stratified sample stored with the generated figure artifacts. The sample contains six failed runs per benchmark and matched the programmatic label in all 30 cases. This check verifies that the rules match visible trace evidence on the sample; it is not a substitute for independent multi-rater annotation.

C. Additional Case Notes

C.1. BrowseComp: Hypothesis Lock-In

One BrowseComp trace asks for an entity satisfying several linked temporal and evidential constraints. A fully linked debate run terminates with an explicitly blocked answer because one required evidence condition remains unsupported. The final reason is no meaningful change, and the vote tally collapses to variants of unknown.

The important detail is that the topology is structurally rich: it assigns a query decomposer, search strategist, document navigator, and answer synthesizer. Yet the trace shows the group converging on the same missing boundary rather than discovering the decisive document. The fatal pivot is not a hallucinated final answer; it is a stagnation state in which the system recognizes that one required criterion is unsupported but has no recovery mechanism beyond additional debate. Debate correctly avoids overclaiming, but it also fails to generate new evidence. This case therefore explains both sides of BrowseComp: unsupported answers are dangerous, but blocked answers can also reveal that coordination has not changed retrieval coverage.

C.2. WorkBench and Finance: Grounding Versus Support

One WorkBench trace shows an operational grounding failure. The tool layer offers calendar operations and a company-directory lookup. The run repeatedly attempts to resolve a required participant, then ends because the required email binding remains missing. The final vote reaches consensus, but consensus is over an unresolved binding. No amount of scheduling logic can create the calendar event without the participant email.

One Finance Agent trace shows the parallel failure in evidential rather than operational form. The trace includes filing search, web search, parsing, and retrieval tools, but the final answer blocks because period-aligned data remain unverified under interface constraints. Some intermediate artifacts contain partial calculations or placeholders, which is exactly the dangerous state for financial QA: the system can produce numeric-looking output before the evidence chain is complete. The fatal pivot to avoid is not only wrong arithmetic; it is allowing partial filings and period-misaligned evidence to support a precise final claim.

D. Additional Figures

E. Artifact Notes

The primary aggregate file is the experiment summary CSV. Task-level metrics are drawn from system-level and task-level metric CSV exports. Per-run diagnoses use evaluation JSON, trajectory markdown, and event-level trace JSONL files. The state-transition diagnostics are generated by the trace-state diagnostics script, which writes the Markov transition, vote-entropy, run-statistic, first-fault, topology-conditioned fault, re-audit sample, and propagation files under the figure artifact directory. The analysis excludes incomplete aggregate cells from topology-level summary claims.

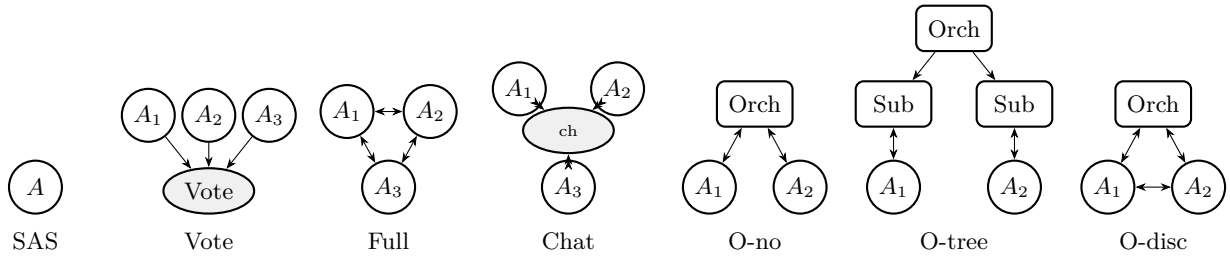

Figure 5. The seven evaluated topologies. Circles are agents, rectangles are orchestrators, ellipses are aggregators. Double-headed arrows indicate bidirectional communication; single-headed arrows indicate one-way delegation. See Table 1 for full names.

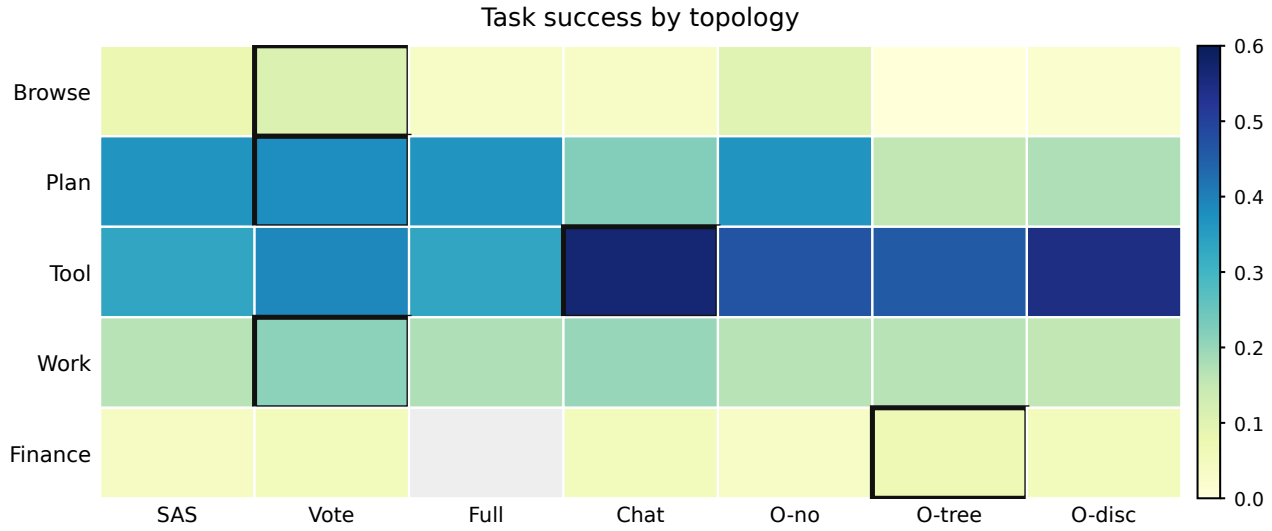

Figure 6. Success rate heatmap by benchmark and topology. The bar chart in Figure 1 is the primary presentation; this heatmap shows the same data in a compact grid for direct cell comparison.

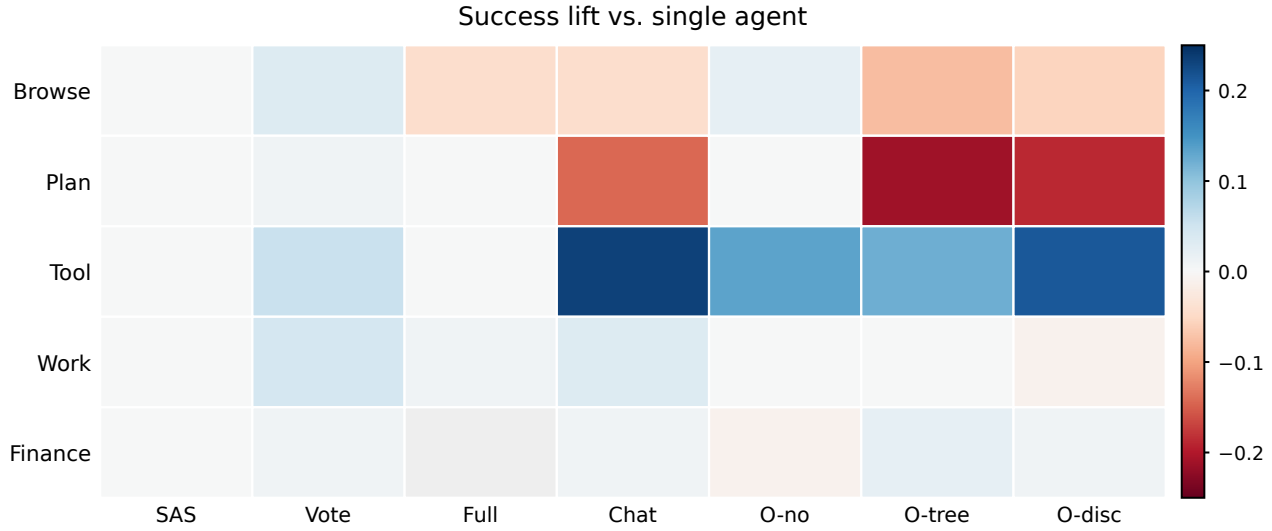

Figure 7. Success lift relative to SAS. Positive effects concentrate in StableToolBench; several debate and orchestrator variants underperform the baseline on BrowseComp and PlanCraft.

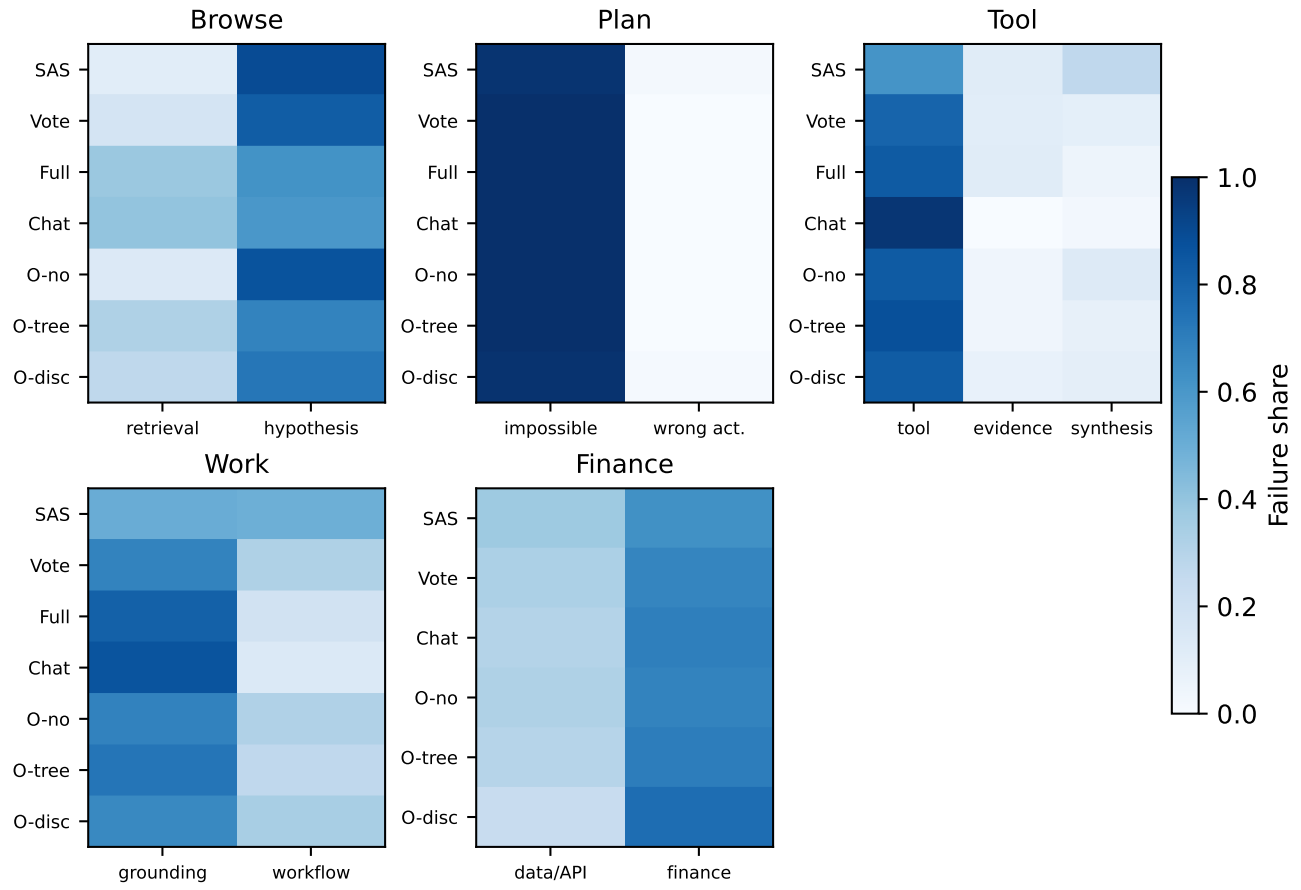

Figure 8. Topology-conditioned first-critical-fault shares. Rows are topologies and columns are first-fault classes within each benchmark. The dominant failure family is usually stable across topology even when success and cost change.

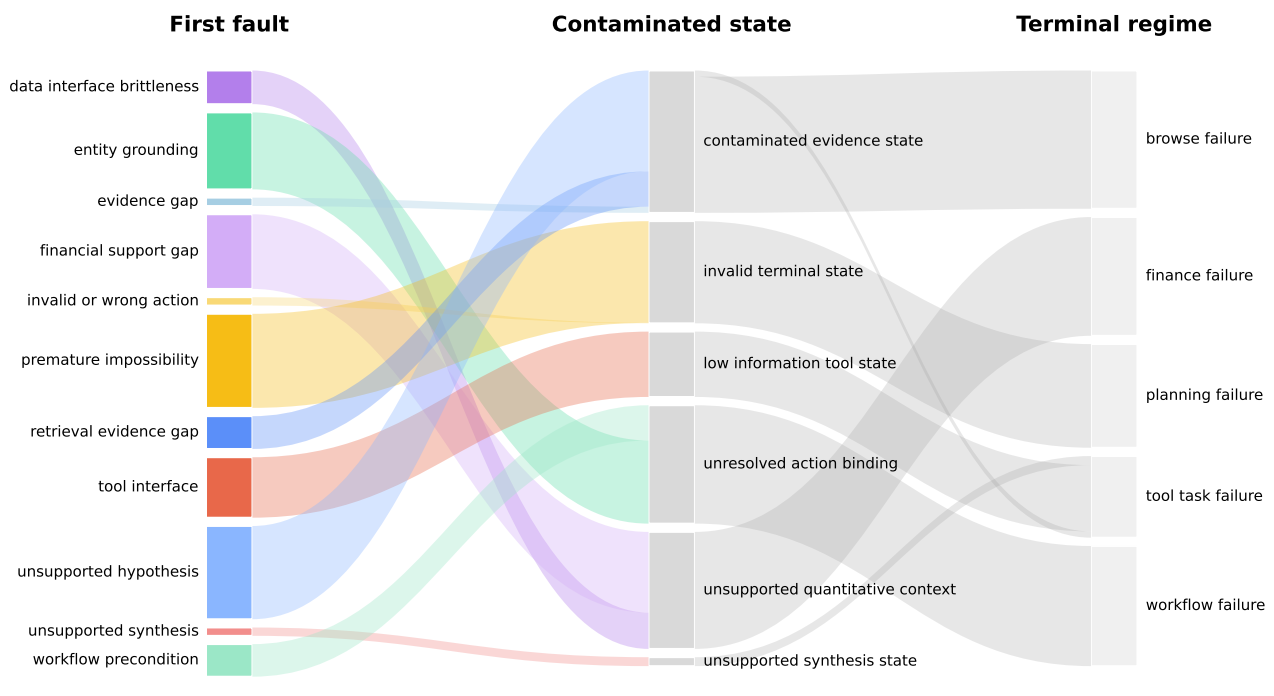

Figure 9. Aggregate diagnostic flow from first-critical-fault to contaminated state and terminal failure regime. The flow should be read as heuristic localization over traces, not as a formal causal graph.

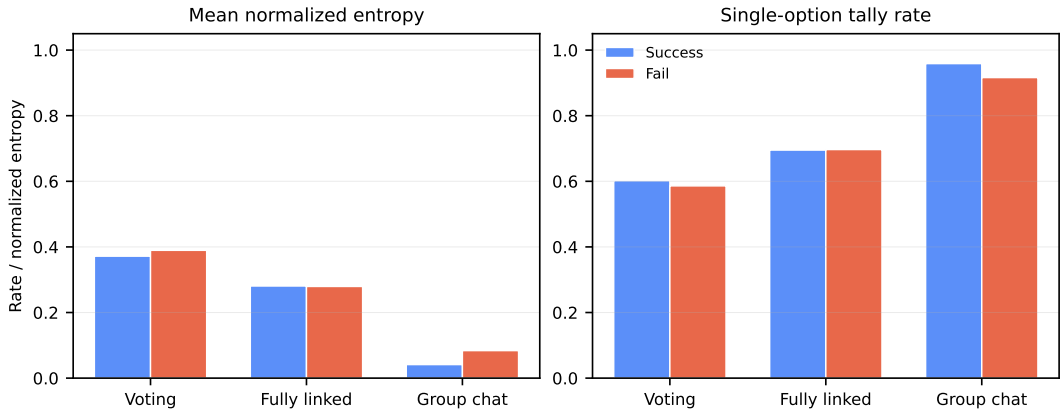

Figure 10. Recorded vote entropy by topology and outcome. Low entropy appears in both successes and failures, so consensus should not be treated as verification.